

Dojo D2 — Tecnologia ponta a ponta

NestJS, Prisma, Next.js, Dexie, TanStack Query e shadcn/ui

DOJO-B · 10 de setembro de 2026

Dojo D2 — Tecnologia ponta a ponta

Rastreabilidade: [issue #11 — Dojo D2, Técnico e demonstrações orientadas](#).

A trilha técnica vai do servidor ao navegador:

1. NestJS, Prisma e PostgreSQL formam a API e a persistência deste documento.
2. Dexie, TanStack Query e shadcn/ui aparecem em `K-modulos-tecnicos.md`.
3. A composição no Next.js e o código para repetição estão em `L-estudo-boilerplates.md`.

As três partes usam o mesmo exemplo didático. Nenhuma define a arquitetura de domínio da aplicação de produção.

D2-S01 · a fatia que vamos atravessar

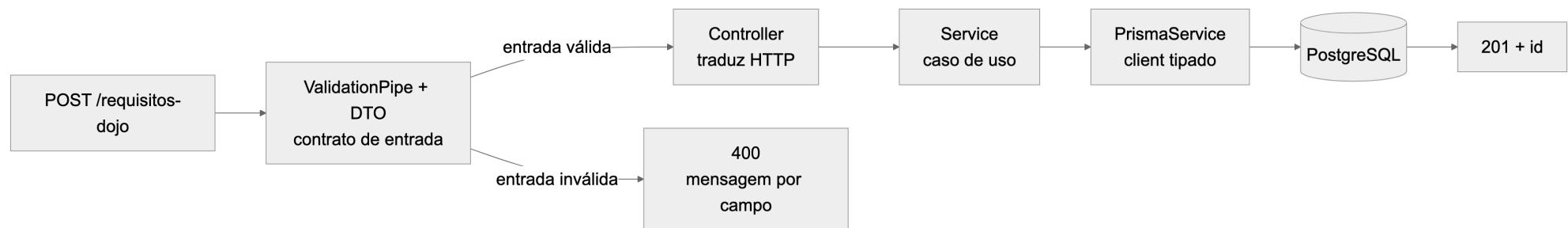


Figura 1 — as seis camadas percorridas por uma requisição válida.

Verificação de sucesso: resposta 201 com id e os cinco campos persistidos. Verificação de erro: resposta 400 quando um campo obrigatório falta, está vazio ou sobra um campo não previsto.

Antes de ler o código, consiga redesenhar esse trajeto e dizer a responsabilidade de cada camada.

D2–S02 · limite de domínio do exercício

[PROPOSTA] O modelo RequisitoDojo é um **artefato de aprendizagem** sobre campos cuja existência já está documentada:

- origem;
- indicador;
- parâmetro de aferição;
- forma de verificação;
- frequência de apuração.

Ele **não decide**:

- se requisito e meta são entidades separadas ou tipos de um agregado;
- se demanda é entidade ou atributo;
- se evidência sem meta é proibida ou aceita como rascunho;
- a base legal de cada finalidade de tratamento de dados.

O nome contém Dojo deliberadamente para que o exemplo não pareça uma decisão de produção.

D2-S03 · NestJS em quatro conceitos

Conceito	Frase de bolso	Onde aparece
Módulo	Declara o conjunto de peças que trabalham juntas.	<code>requisitos-dojo.module.ts</code>
Controlador	Recebe HTTP e delega o caso de uso.	<code>requisitos-dojo.controller.ts</code>
Provedor	Implementa trabalho reutilizável e pode ser injetado.	<code>requisitos-dojo.service.ts</code> , <code>prisma.service.ts</code>
Injeção	O construtor pede a dependência; o contêiner fornece.	construtores de controller e service

Uma classe só fica disponível para injeção quando o provider está declarado no módulo correto e os módulos necessários estão importados.

Fontes: [NestJS Modules](#), [Controllers](#), [Providers](#), acesso em 10/09/2026.

D2–S04 · validação em três opções

```
TS
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
  }),
);
```

- `whitelist`: remove propriedades sem decorador de validação.
- `forbidNonWhitelisted`: em vez de apenas removê-las, devolve erro.
- `transform`: converte o corpo para a classe do DTO quando possível.

Mensagem por decorador:

```
TS
@IsEmpty({ message: 'origem é obrigatória' })
```

Um corpo com campo `Inventado` deve ser rejeitado. O contrato não salva entrada inesperada silenciosamente.

Fontes: [NestJS Validation](#), [class-validator README](#), acesso em 10/09/2026.

D2–S05 · Prisma em quatro conceitos

Conceito	Frase de bolso	Onde aparece
Schema	Declara datasource, gerador e modelos.	<code>prisma/schema.prisma</code>
Migration	Versão SQL revisável que muda o banco.	<code>prisma/migrations/<timestamp>_init_requisito_dojo/migration.sql</code>
Client	API TypeScript gerada a partir do schema.	<code>src/generated/prisma/</code>
Adapter	Conecta o client atual ao driver PostgreSQL.	construtor de <code>PrismaService</code>

[DOCUMENTADO] Em Prisma 7, `migrate dev` não executa `generate` implicitamente. O `dojo` roda os dois comandos de forma explícita.

Schema é intenção, migration é histórico aplicável e client é código derivado. Os três artefatos não são sinônimos.

Fontes: [Prisma Client generator](#), [Prisma migrate dev](#), acesso em 10/09/2026.

D2–S06 · dev e deploy não são intercambiáveis

Comando	Uso
<code>prisma migrate dev</code>	Desenvolvimento: detecta divergência, usa banco sombra, cria e aplica migration; pode solicitar reset.
<code>prisma migrate deploy</code>	Homologação/produção: aplica migrations pendentes; não procura divergência, não gera artefatos e não usa banco sombra.

Neste dojo local:

```
BASH
npx prisma migrate dev --name init_requisito_dojo
npx prisma generate
```

Este material não executa deploy. A distinção existe para impedir o uso do comando errado fora do desenvolvimento local.

Fontes: [Prisma migrate dev](#), [Prisma migrate deploy](#), acesso em 10/09/2026.

D2-S07 · versões do ambiente didático

[DOCUMENTADO] Consulta em 10/09/2026:

Componente	Versão usada
Node.js LTS	24.21.0
@nestjs/cli	12.0.0
@nestjs/core	12.0.1
@nestjs/config	12.0.0
Prisma CLI e client	7.10.0
@prisma/adapters-pg	7.10.0
PostgreSQL	imagem postgres:18.6
class-validator	0.15.1
class-transformer	0.5.1
Jest	30.5.1
TypeScript	6.0.3

Atenção: o dist-tag latest do pacote prisma apontava para 8.0.0-rc.13, enquanto @prisma/client apontava para 7.10.0. O ambiente não mistura uma versão candidata com um client estável; ambos ficam fixados em 7.10.0.

Se uma versão diferente aparecer, compare o lockfile e a documentação da versão. Não misture APIs de versões principais diferentes.

D2–S08 · de onde vem o starter

Este bloco é registro, não tarefa. A pasta `api/` já chega pronta na branch; o que segue documenta como ela foi produzida, para que você saiba o que cada escolha significa e possa reproduzi-la em outro contexto.

O scaffold veio de:

```
BASH
npx @nestjs/cli@12.0.0 new api \
  --package-manager npm \
  --strict \
  --skip-git \
  --no-observe
```

Com **CJS (CommonJS)** na pergunta de formato de módulos: é a opção que cria o scaffold com Jest e preserva a stack decidida. Como os pacotes do NestJS 12 são ESM, `api/` também configura o Jest para transformar TypeScript como ESM e executá-lo com `--experimental-vm-modules`. O aviso experimental do Node é esperado.

As dependências fixadas foram:

```
BASH
npm install \
  @nestjs/common@12.0.1 \
  @nestjs/core@12.0.1 \
  @nestjs/platform-express@12.0.1 \
  @nestjs/config@12.0.0 \
  class-validator@0.15.1 \
  class-transformer@0.5.1 \
  @prisma/client@7.10.0 \
  @prisma/adapters-pg@7.10.0 \
  pg@8.23.0 \
  dotenv@17.4.2
npm install --save-dev \
  prisma@7.10.0 \
  @types/pg@8.23.1 \
  supertest@7.2.2 \
  @types/supertest@7.2.1
```

O que você roda é outra coisa:

```
BASH
cd api
npm ci
```

`npm ci` instala exatamente o que o `package-lock.json` registra. `npm install` pode resolver versões diferentes das validadas; num exercício em que todos precisam ver o mesmo comportamento, essa diferença custa caro. Se `npm ci` reclamar de lockfile fora de sintonia, o problema é o Node — confira a versão antes de qualquer outra hipótese.

`--skip-git` impedia um repositório aninhado durante a preparação. `tsconfig.json`, `jest.config.ts` e `test/jest-e2e.json` **já estão aplicados em api/** com a compatibilidade validada: sem eles, o scaffold CJS compila a aplicação, mas o Jest falha ao carregar os pacotes ESM do NestJS 12.

Fontes: [Nest CLI usage](#), [NestJS migration guide](#), acesso em 10/09/2026.

D2–S09 · PostgreSQL local atual

Arquivo api/compose.yaml:

```
YAML
services:
  db:
    image: postgres:18.6
    environment:
      POSTGRES_USER: cybersetor
      POSTGRES_PASSWORD: dojo-local
      POSTGRES_DB: cybersetor_dojo
    ports:
      # Porta do host. Se 5432 já estiver ocupada na sua máquina, defina
      # DB_PORT no .env (e ajuste a porta em DATABASE_URL junto).
      - "${DB_PORT:-5432}:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U cybersetor -d cybersetor_dojo"]
      interval: 2s
      timeout: 3s
      retries: 15
    volumes:
      - pg_dojo:/var/lib/postgresql

volumes:
  pg_dojo:
```

Não há chave version: no Compose atual ela é apenas informativa, está obsoleta e pode gerar aviso. Na imagem PostgreSQL 18+, o volume deve ser montado em /var/lib/postgresql.

O healthcheck informa prontidão, mas não substitui a conferência de accepting connections antes da migration.

Fontes: [Docker Compose version and name](#), [Compose startup order](#), [imagem oficial PostgreSQL](#), acesso em 10/09/2026.

D2–S10 · configuração local

Copie o exemplo versionado e ajuste se precisar:

```
BASH
cd api
cp .env.example .env
```

O conteúdo de `api/.env`:

```
DOTENV
DB_PORT=5432
DATABASE_URL="postgresql://cybersetor:dojo-local@localhost:5432/cybersetor_dojo?schema=public"
PORT=3001
```

`DB_PORT` é a porta do PostgreSQL **na sua máquina**. Só mude se a 5432 já estiver ocupada por outro projeto — e, se mudar, troque a porta em `DATABASE_URL` também: as duas precisam combinar. O erro de porta ocupada é `Bind for 127.0.0.1:5432 failed: port is already allocated`, e ele aparece no `docker compose up`, não no Prisma.

Arquivo `api/prisma.config.ts`:

```
TS
import 'dotenv/config';
import { defineConfig, env } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
});
```

`.env` é local e não deve ser publicado. Estas credenciais são descartáveis; credenciais reais nunca entram na documentação nem no histórico.

Fonte: [Prisma config reference](#), acesso em 10/09/2026.

D2–S11 · schema do exercício

Arquivo `api/prisma/schema.prisma`:

```
PRISMA
generator client {
  provider      = "prisma-client"
  output        = "../src/generated/prisma"
  moduleFormat  = "cjs"
}

datasource db {
  provider = "postgresql"
}

// Artefato didático. Não define a fronteira de produção entre requisito e meta.
model RequisitoDojo {
  id          String   @id @default(uuid())
  origem      String
  indicador   String
  parametroAfericao String
  formaVerificacao String
  frequenciaApuracao String
  criadoEm    DateTime @default(now())
}
```

`moduleFormat = "cjs"` alinha o client gerado ao scaffold CommonJS. O output explícito é obrigatório no gerador atual.

Fontes: [Prisma Client generator](#), [NestJS Prisma recipe](#), acesso em 10/09/2026.

D2–S12 · gerar a estrutura Nest

Aqui você constrói: a base **não** tem requisitos-dojo/. A partir de api/:

```
BASH
npx nest generate module requisitos-dojo
npx nest generate controller requisitos-dojo --no-spec
npx nest generate service requisitos-dojo --no-spec
npx nest generate service prisma --flat --no-spec
```

Arquivos esperados:

```
TEXT
src/
├── app.module.ts
├── main.ts
├── prisma.service.ts
├── requisitos-dojo/
│   ├── dto/create-requisito-dojo.dto.ts
│   ├── requisitos-dojo.controller.ts
│   ├── requisitos-dojo.module.ts
│   └── requisitos-dojo.service.ts
```

Depois da geração, confira `app.module.ts` e `requisitos-dojo.module.ts`. O CLI cria a classe do service; o caso de uso é acrescentado a seguir.

Fonte: [Nest CLI usage](#), acesso em 10/09/2026.

D2–S13 · o Prisma como provider injetável

Arquivo `api/src/prisma.service.ts`:

```
TS
import { Injectable, OnModuleDestroy } from '@nestjs/common';
import { PrismaPg } from '@prisma/adapter-pg';
import { PrismaClient } from '../generated/prisma/client';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleDestroy {
  constructor() {
    const connectionString = process.env.DATABASE_URL;

    if (!connectionString) {
      throw new Error('DATABASE_URL não foi definida');
    }

    // Prisma 7 usa um adapter para conectar o client ao driver PostgreSQL.
    const adapter = new PrismaPg({ connectionString });
    super({ adapter });
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}
```

[DOCUMENTADO] A documentação atual de NestJS com Prisma continua mostrando um provider que estende `PrismaClient`; em Prisma 7, ele é construído com o adapter do banco.

`PrismaService` é a porta do módulo para o client. Não crie um `PrismaClient` novo dentro de cada método.

Fonte: [Prisma, NestJS guide](#), acesso em 10/09/2026.

D2–S14 · DTO com mensagens úteis

Criar `api/src/requisitos-dojo/dto/create-requisito-dojo.dto.ts`:

```
TS
import { IsNotEmpty, IsString } from 'class-validator';

export class CreateRequisitoDojoDto {
  @IsString({ message: 'origem deve ser texto' })
  @IsNotEmpty({ message: 'origem é obrigatória' })
  origem!: string;

  @IsString({ message: 'indicador deve ser texto' })
  @IsNotEmpty({ message: 'indicador é obrigatório' })
  indicador!: string;

  @IsString({ message: 'parametroAfericao deve ser texto' })
  @IsNotEmpty({ message: 'parametroAfericao é obrigatório' })
  parametroAfericao!: string;

  @IsString({ message: 'formaVerificacao deve ser texto' })
  @IsNotEmpty({ message: 'formaVerificacao é obrigatória' })
  formaVerificacao!: string;

  @IsString({ message: 'frequenciaApuracao deve ser texto' })
  @IsNotEmpty({ message: 'frequenciaApuracao é obrigatória' })
  frequenciaApuracao!: string;
}
```

Validação de presença e validação de tipo ficam separadas para que a resposta explique o erro real.

D2–S15 · service executa o caso mínimo

Arquivo `api/src/requisitos-dojo/requisitos-dojo.service.ts`:

```
TS
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../prisma.service';
import { CreateRequisitoDojoDto } from '../dto/create-requisito-dojo.dto';

@Injectable()
export class RequisitosDojoService {
  constructor(private readonly prisma: PrismaService) {}

  listar() {
    return this.prisma.requisitoDojo.findMany({
      orderBy: { criadoEm: 'desc' },
    });
  }

  criar(dto: CreateRequisitoDojoDto) {
    // O tipo de `data` é derivado do schema; não duplicamos um modelo manual.
    return this.prisma.requisitoDojo.create({ data: dto });
  }
}
```

O service não conhece HTTP. Ele recebe um comando validado e pede persistência ao Prisma.

D2–S16 · controller traduz HTTP para o service

Arquivo `api/src/requisitos-dojo/requisitos-dojo.controller.ts`:

```
TS
import { Body, Controller, Get, Post } from '@nestjs/common';
import { CreateRequisitoDojoDto } from '../dto/create-requisito-dojo.dto';
import { RequisitosDojoService } from '../requisitos-dojo.service';

@Controller('requisitos-dojo')
export class RequisitosDojoController {
  constructor(private readonly requisitos: RequisitosDojoService) {}

  @Get()
  listar() {
    return this.requisitos.listar();
  }

  @Post()
  criar(@Body() dto: CreateRequisitoDojoDto) {
    return this.requisitos.criar(dto);
  }
}
```

O controller fica fino de propósito: responde “qual rota?” e “para quem delega?”.

D2–S17 · módulo liga as peças

Arquivo `api/src/requisitos-dojo/requisitos-dojo.module.ts`:

```
TS
import { Module } from '@nestjs/common';
import { PrismaService } from '../prisma.service';
import { RequisitosDojoController } from './requisitos-dojo.controller';
import { RequisitosDojoService } from './requisitos-dojo.service';

@Module({
  controllers: [RequisitosDojoController],
  providers: [RequisitosDojoService, PrismaService],
})
export class RequisitosDojoModule {}
```

Arquivo `api/src/app.module.ts`:

```
TS
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { RequisitosDojoModule } from './requisitos-dojo/requisitos-dojo.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    RequisitosDojoModule,
  ],
})
export class AppModule {}
```

Se a injeção falhar, leia a mensagem e confira primeiro `providers` e `imports`.

D2–S18 · pipe global fecha o contrato de entrada

Arquivo `api/src/main.ts`:

```
TS
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.enableCors({ origin: 'http://localhost:3000' });

  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );

  await app.listen(Number(process.env.PORT ?? 3001));
}

void bootstrap();
```

Sem o pipe global, os decoradores do DTO não entram automaticamente no fluxo HTTP.

D2–S19 · subir, esperar, migrar, gerar

```
BASH
docker compose up -d
docker compose ps
docker compose exec db pg_isready -U cybersetor -d cybersetor_dojo
npx prisma migrate dev --name init_requisito_dojo
npx prisma generate
npm run build
npm run start:dev
```

Saídas que liberam o próximo passo:

- `docker compose ps`: serviço db como healthy.
- `pg_isready`: `accepting connections`.
- `migration`: nova pasta em `prisma/migrations/` e migration aplicada.
- `generate`: client escrito em `src/generated/prisma/`.
- `build`: processo termina sem erro.

Só rode a migration depois de `accepting connections`. Se o banco ainda estiver subindo, diagnostique o ambiente; não altere código para mascarar falta de prontidão.

D2–S20 · provar sucesso e falha

Sucesso:

```
BASH
curl -i -X POST http://localhost:3001/requisitos-dojo \
  -H 'Content-Type: application/json' \
  -d '{
    "origem":"Edital",
    "indicador":"participantes com presença registrada",
    "parametroAfericao":"quantidade por atividade",
    "formaVerificacao":"lista de presença",
    "frequenciaApuracao":"por atividade"
  }'
```

Esperado: HTTP/1.1 201 e JSON com id.

Falha observável:

```
BASH
curl -i -X POST http://localhost:3001/requisitos-dojo \
  -H 'Content-Type: application/json' \
  -d '{"origem":"","campoInventado":true}'
```

Esperado: HTTP/1.1 400 com mensagens sobre campos obrigatórios e propriedade não permitida.

Listagem usada pelo módulo de TanStack Query:

```
BASH
curl -i http://localhost:3001/requisitos-dojo
```

Esperado: HTTP/1.1 200 e um array JSON, incluindo o item criado.

Os valores enviados são apenas dados didáticos e não afirmam regra de negócio do Instituto.

D2–S21 · verificação automatizada posterior

O arquivo completo está em `gabarito/api/test/requisitos-dojo.e2e-spec.ts`. Ele limpa apenas a tabela descartável do dojo e prova três comportamentos. Nos trechos abaixo, `dtoValido` é o mesmo JSON completo apresentado em D2–S20:

```
TS
it('rejeita entrada incompleta e propriedade desconhecida', async () => {
  await request(app.getHttpServer())
    .post('/requisitos-dojo')
    .send({ origem: '', campoInventado: true })
    .expect(400);
});

it('persiste uma entrada válida e devolve 201 com id', async () => {
  const response = await request(app.getHttpServer())
    .post('/requisitos-dojo')
    .send(dtoValido)
    .expect(201);

  expect(response.body.id).toEqual(expect.any(String));
});

it('lista por GET o registro criado', async () => {
  const response = await request(app.getHttpServer())
    .get('/requisitos-dojo')
    .expect(200);

  expect(response.body).toEqual(
    expect.arrayContaining([expect.objectContaining({ origem: 'Edital' })]),
  );
});
```

Executar:

```
BASH
npm run test:e2e -- requisitos-dojo.e2e-spec.ts
```

Resultado validado no ambiente preparado: **3 testes passando**. O aviso `ExperimentalWarning: VM Modules` é esperado pela ponte Jest/NestJS 12 e não representa falha. Estes testes confirmam o contrato depois da implementação; portanto, não demonstram TDD.

Fonte: [NestJS testing](#), acesso em 10/09/2026.

D2–S22 · percorrer a fatia completa

Leia a implementação na ordem em que a requisição atravessa o sistema:

1. confira o corpo enviado e o contrato do DTO;
2. localize o `ValidationPipe` e explique como ele produz o `400`;
3. siga do controller para o service;
4. encontre a chamada tipada do Prisma;
5. relacione o model à migration aplicada no PostgreSQL;
6. envie uma entrada inválida e leia as mensagens;
7. envie uma entrada válida e localize o registro persistido;
8. use `GET /requisitos-dojo` e confirme que a leitura devolve o registro.

O objetivo é explicar a responsabilidade de cada arquivo, não decorar o código.

D2–S23 · exercício de reconstrução

Partindo do scaffold sem o módulo de domínio, reconstrua estas seis etapas:

1. Schema e configuração.
2. Banco saudável e migration.
3. Provider Prisma e módulo.
4. DTO e pipe global.
5. Controller e service.
6. Requisições de sucesso e erro.

Após cada etapa, o projeto deve continuar explicável. Se surgir um erro, registre o comando, a saída exata e a primeira hipótese antes de alterar outro arquivo.

D2–S24 · teste automatizado e TDD não são sinônimos

O teste de D2–S21 foi escrito para verificar uma implementação já existente. Ele prova parte do contrato, mas não demonstra TDD.

Em TDD, a ordem observável é:

1. escrever um teste relevante que falha;
2. implementar o mínimo para fazê-lo passar;
3. refatorar preservando o teste verde.

Ter Jest e Supertest instalados não comprova que essa sequência foi praticada. A adoção de TDD só pode ser declarada quando o trabalho registrar os três estados.

D2–S25 · checklist técnico da fatia

- ☐ banco healthy e aceitando conexões;
- ☐ migration versionada em pasta própria;
- ☐ client gerado explicitamente;
- ☐ build sem erro;
- ☐ entrada inválida devolvendo 400;
- ☐ entrada válida devolvendo 201 com id.
- ☐ leitura devolvendo GET 200 com o item criado;
- ☐ suíte E2E com três testes passando.

Se algum item falhar, registre o primeiro item não satisfeito e o erro exato. Não declare a fatia concluída por aproximação.

Fontes

- [NestJS CLI](#), acesso em 10/09/2026.
- [NestJS migration guide](#), acesso em 10/09/2026.
- [NestJS validation](#), acesso em 10/09/2026.
- [Prisma com NestJS](#), acesso em 10/09/2026.
- [Prisma Migrate](#), acesso em 10/09/2026.
- [Imagem oficial PostgreSQL](#), acesso em 10/09/2026.
- [Docker Compose](#), acesso em 10/09/2026.